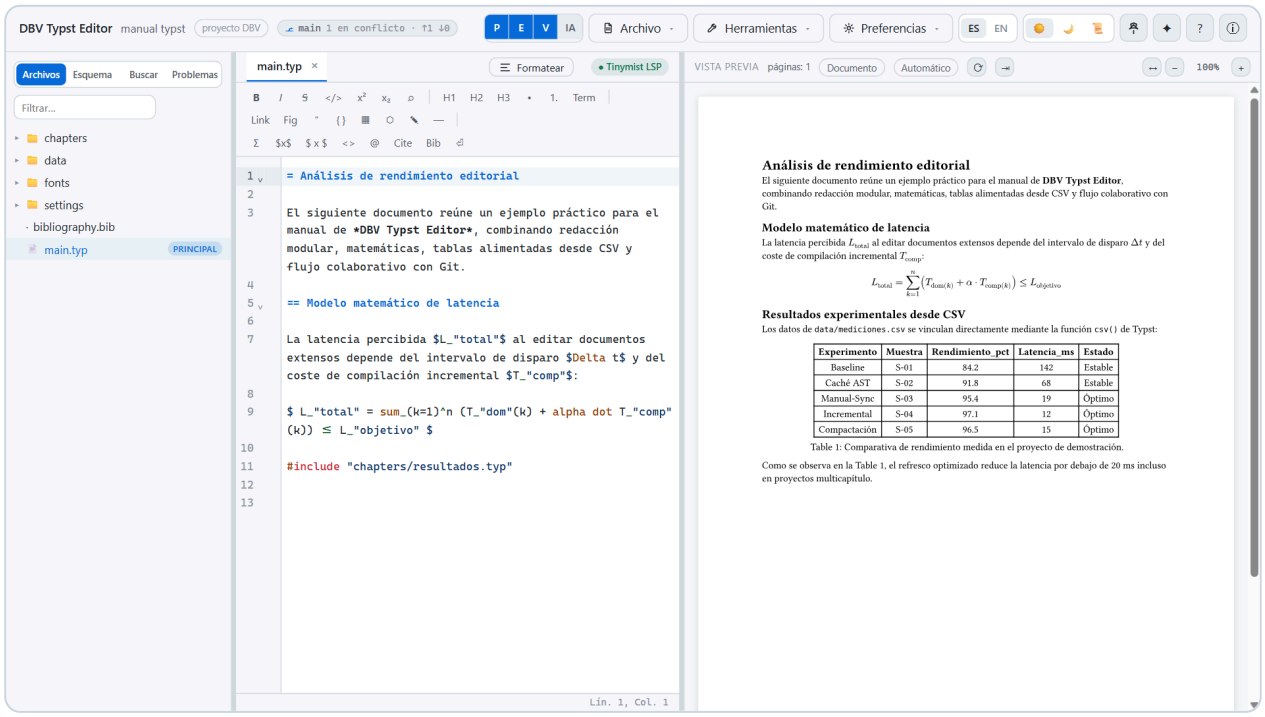
User Manual · English · v0.15.0

DBV Typst Editor

Complete User Manual: academic & technical authoring, full menu reference, Git collaboration, visual tools, assistants, and optional AI



Author: David Bueno Vallejo
Typst: 0.15.1 (embebido / bundled)
License: MIT License
2026

Repository: github.com/davidbuenov/dbv-typst-editor · Web

Table of contents

1. What is DBV Typst Editor . . . 1
   - 1.1. What is new in v0.15.0 . . . 1
2. Getting Started & Project Launcher . . . 2
3. Workspace Layout & Four Independent Panels . . . 4
4. Complete Menu Reference . . . 5
   - 4.1. File Menu . . . 5
   - 4.2. Tools Menu . . . 5
   - 4.3. Preferences Menu . . . 6
5. High-Performance Authoring, Toolbar & Vim Mode . . . 8
   - 5.1. Context-Aware Toolbar & BibTeX Citations . . . 8
   - 5.2. Optional Vim Mode, Relative Lines & Zero-Latency Manual Refresh . . . 8
6. Visual Customization: Themes, Colors & WCAG Contrast . . . 10
7. Collaborative Workflows with Git . . . 11
   - 7.1. Cloning & Organizing a Shared Team Project . . . 11
   - 7.2. Header Git Pill & Status Popover . . . 11
   - 7.3. Visual Block-by-Block Conflict Resolver & Disk Diff Dialog . . . 12
8. Visual Tools & Assistants with Concrete Typst Examples . . . 13
   - 8.1. Visual Diagram Editor (`cetz`) . . . 13
   - 8.2. Visual Equation Editor & LaTeX Converter (`MiTeX`) . . . 13
   - 8.3. Sequence Diagrams (`chronos`) . . . 14
   - 8.4. Gantt Charts (`gantty`) . . . 14
   - 8.5. Kanban Boards (`kantan`) . . . 14
   - 8.6. DOT / Graphviz (`diagraph`) . . . 14
   - 8.7. CSV / TSV Data Viewer & Live `csv()` Tables . . . 14
   - 8.8. Python Script Runner & Embedded JavaScript (`jogs`) . . . 15
9. VS Code Snippets, Sublime Import & Custom Keybindings . . . 16
10. Project Fonts, Typst Universe & Advanced Terminal . . . 17
    - 10.1. Self-Contained Fonts in `fonts/` . . . 17
    - 10.2. Typst Universe Templates & Packages . . . 17
    - 10.3. Integrated Typst Terminal & MCP Server . . . 17
11. Complete AI Assistant Guide (Local, Agents, or Cloud) . . . 19
    - 11.1. Track 1: Local AI on Your Machine (Total Privacy) . . . 19
    - 11.2. Track 2: Cloud Providers with an API Key . . . 20
    - 11.3. Track 3: CLI Agents Using Your Subscription (ACP) . . . 20
    - 11.4. Project Capabilities & Verified Change Proposals . . . 21
    - 11.5. Privacy Summary & Quick Troubleshooting . . . 21
    - 11.6. DBV's MCP Server for External Coding Agents (`Tools › MCP server…`) . . . 21
12. Credits & Acknowledgments . . . 23
    - 12.1. Author & Collaborators . . . 23
    - 12.2. Community Contributors (`v0.15.0`) . . . 23
    - 12.3. Open-Source Projects Powering DBV . . . 23

1.

What is DBV Typst Editor

DBV Typst Editor is a lightweight, self-contained desktop workspace (Rust + Tauri v2, without Electron) purpose-built for authoring bachelor's and master's theses, doctoral dissertations, scientific papers, technical reports, 16:9 slide decks, and resumes using the **Typst** typesetting system.

Unlike a generic code IDE, the application embeds the official Typst compiler in memory alongside the **Tinymist LSP** language server: write on the left and watch typeset pages render on the right in real time, complete with live compiler diagnostics, BibTeX citation assistants, visual diagram/equation editors, and one-click export to PDF, PNG, or a portable `.dbvt` archive.

> **Zero lock-in and 100% standard Typst**
>
> Every project created in DBV Typst Editor is an ordinary Typst project. You can compile it with `typst compile main.typ` from any terminal, collaborate over Git with authors using any other editor, or package the entire workspace including custom fonts into a portable `.dbvt` file.

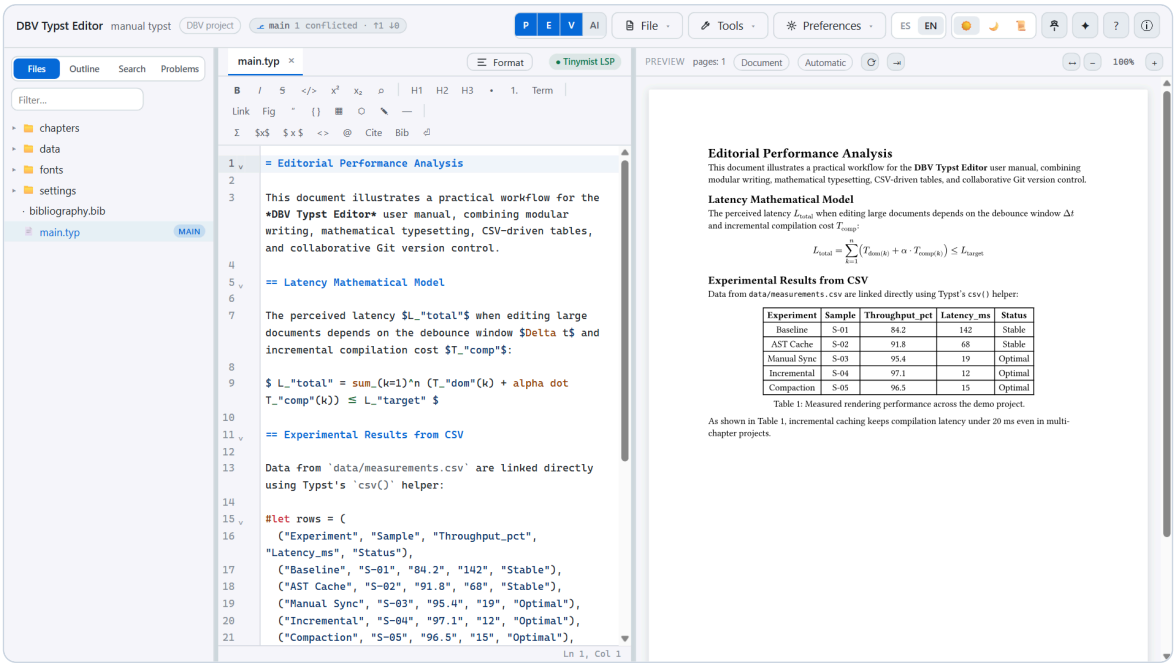


Figure 1: Main split workspace in DBV Typst Editor v0.15.0: project tree, syntax-highlighted Typst editor, and real-time SVG preview.

1.1. What is new in v0.15.0

- **10 curated themes + WCAG AA Custom Theme editor:** Dark (with tuned syntax contrast), Light, Sepia, High contrast, Nord, Dracula, Catppuccin Mocha, Catppuccin Latte, Solarized Dark, and Gruvbox Dark, plus configurable header quick-switch slots.
- **Zero-latency Manual Refresh mode:** debounced LSP notifications and incremental open-tab refreshes keep typing fluid even in multi-hundred-page theses.
- **Optional Vim mode & relative line numbers:** lazy-loaded modal editing (`NORMAL`, `INSERT`, `VISUAL`, `V-LINE`, `V-BLOCK`, `REPLACE`) showing a live footer pill, `:w` save mapping, and popup-safe `Escape` handling.
- **Full collaborative Git workflow:** header status pill and popover, pre-pull local conflict guard, visual block-by-block merge conflict resolver, and built-in repository cloning.
- **Scientific & data authoring assistants:** interactive `cetz` Diagram editor, visual Equations with `MiTeX` LaTeX import, `chronos` Sequence diagrams, `gantty` Gantt charts, `kantan` Kanban boards, `diagraph` DOT graphs, CSV/TSV Data viewer, Python script runner, and embedded JS via `jogs`.

2.

Getting Started & Project Launcher

Opening the application without an active workspace shows the **project launcher** (Figure 2). From here you can start from a guided template, open an existing directory or loose `.typ` file, import a `.dbvt` package, or clone a Git repository.

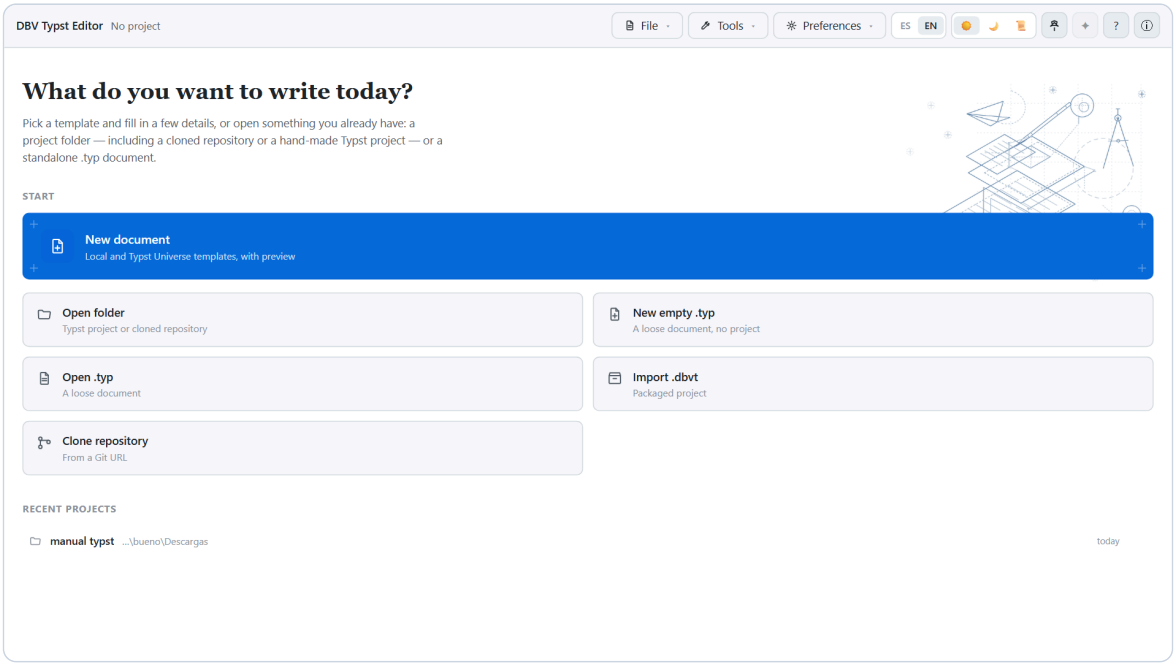


Figure 2: Initial project launcher: academic templates, open folder or standalone file, .dbvt import, and direct Git cloning.

Launcher action	When to use it
New document	Opens the gallery featuring **8 built-in academic templates** (Scientific paper, Bachelor Thesis, Master Thesis, PhD Dissertation, Technical report, Presentation 16:9, Curriculum Vitae, and Blank project) alongside curated **Typst Universe** community templates. Built-in templates collect author, institution, language, and cover metadata through a guided form.
Open folder	Loads any existing folder containing `.typ` files (such as a cloned Git repo or hand-crafted project). If `settings/dbv-project.toml` is present, its shared main entrypoint is respected automatically.
New empty .typ	Creates a standalone scratch `.typ` document outside a project folder for quick experiments or notes.
Open .typ	Opens and previews an individual `.typ` file without requiring a project directory.
Import .dbvt	Extracts a self-contained `.dbvt` archive received from a collaborator or advisor, restoring chapters, images, bibliography, and embedded fonts.
Clone repository	Clones a remote Git repository into a local folder with progress feedback and opens the resulting project ready for collaboration.

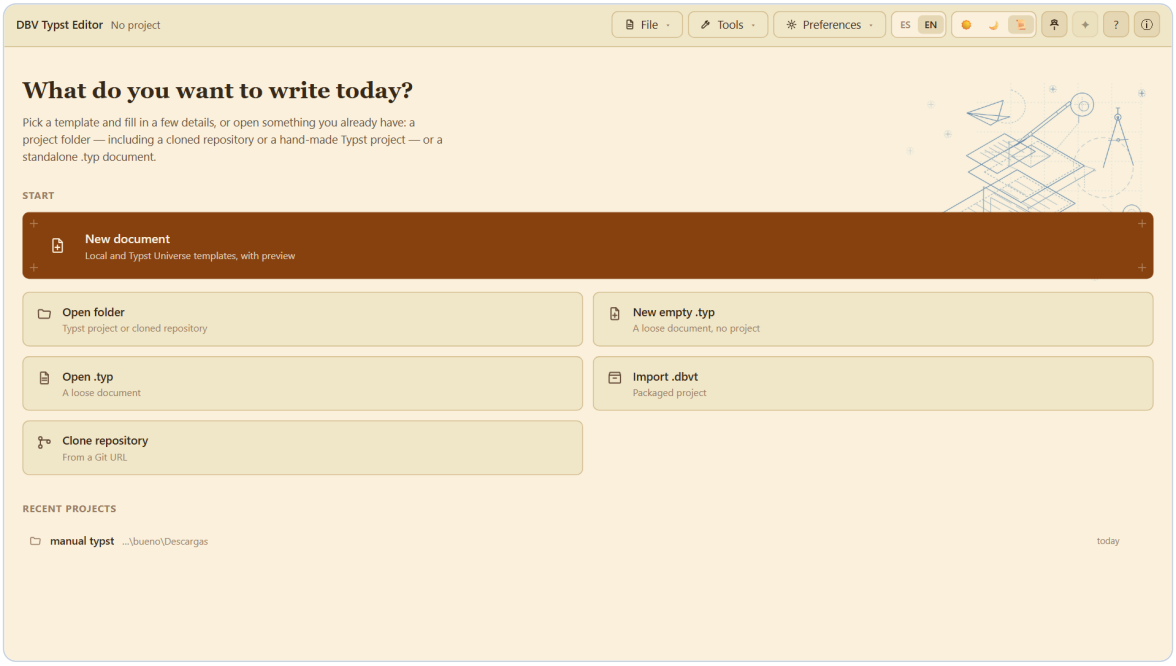


Figure 3: Launcher switched to the warm Sepia theme via the header quick-switch controls, with instant ES/EN UI language switching.

3.

Workspace Layout & Four Independent Panels

The top header unifies the **P / E / V / AI** panel toggles, the **File**, **Tools**, and **Preferences** menus, the **ES / EN** language selector, three quick-theme buttons, the Typst terminal >_, Typst Universe ✦, interactive Help ?, and **About** ℹ.

Button	Panel and primary role
P — Project	Left sidebar housing four tabs: **Files** (project tree with `MAIN` badge on the root entrypoint), **Outline** (live section hierarchy), **Search** (project-wide search and replace with Rust regex), and **Problems** (compiler errors and warnings grouped by file).
E — Editor	Center writing workspace with draggable tabs, context-aware formatting toolbar, theme-adapted syntax highlighting, Tinymist LSP completions, and a live footer showing `Ln X, Col Y`, selection count, and active Vim mode.
V — Preview	Rendered page viewer featuring compilation scope (**Document** root vs active **File**), refresh mode (**Automatic** vs **Manual**), zoom levels, Fit width ⇔, cursor sync pin, preview search `Ctrl/Cmd+F`, and double-click source jumping.
AI — Assistant	Optional right-hand AI panel. Appears only when an AI provider is configured and a project is open. Displays automatic context chips, thread switcher, verified proposed-change cards, and a **Show in preview** action.

Panel splitters resize smoothly by dragging; double-clicking any splitter restores default widths. When multiple floating popovers or dialogs are open (Git, Preview search, Universe, Typst docs, Diagram editor), clicking any popup automatically brings it to the front.

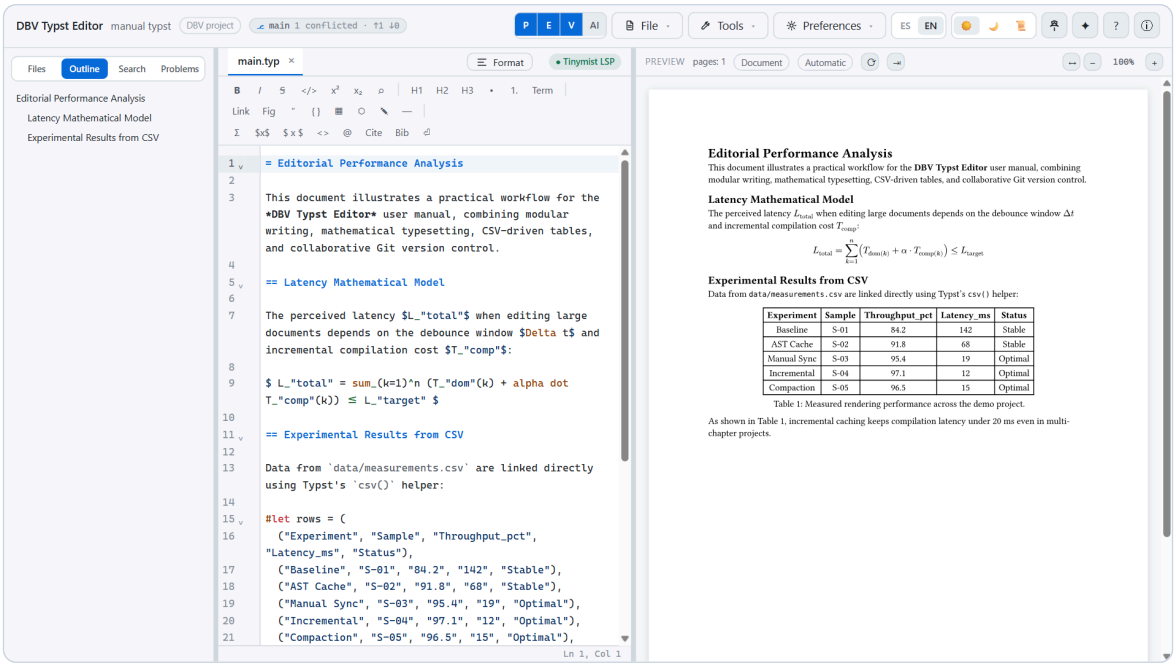


Figure 4: Outline tab displaying live document headings; clicking any entry jumps directly to that page in the typeset preview.

4.

Complete Menu Reference

Every capability of DBV Typst Editor is organized across three header menus that switch seamlessly on hover/click and support full keyboard navigation (↑, ↓, `Home`, `End`, `Escape`).

4.1. File Menu

Menu item (File)	Behavior and details
New blank project	Creates a minimal project folder containing an empty `main.typ` ready to write with no form wizard.
New document...	Opens the combined gallery of built-in academic templates and Typst Universe community templates.
Save (`Ctrl/Cmd+S`)	Writes the active document to disk. If the file changed externally while open, displays the side-by-side disk diff resolver before overwriting.
Save as... (`Ctrl/Cmd+Shift+S`)	Saves the current document under a new filename or path.
Export PDF...	Compiles the project entrypoint to a publication-ready `.pdf` with internal links, PDF outline bookmarks, and embedded fonts.
Export PNG...	Exports the page currently displayed in the preview viewport as a crisp `.png` image.
Set as main document	Designates the active `.typ` file as the compilation root and persists that setting inside `settings/dbv-project.toml` so every collaborator compiles from the same root.
Add chapter...	Creates a new modular `.typ` chapter inside `chapters/` and inserts its `#include` statement into the main document.
Open project folder... (`Ctrl/Cmd+O`)	Switches to another Typst project directory on disk.
Open .typ document...	Opens a standalone `.typ` document outside a project workspace.
Import project (.dbvt)...	Unpacks a portable `.dbvt` archive into a chosen destination directory and opens it.
Export project (.dbvt)...	Bundles chapters, images, bibliography, `settings/dbv-project.toml`, and `fonts/` into a single `.dbvt` ZIP archive.
Reveal in Explorer / Finder	Opens the active project folder in the OS file manager.
Close project	Prompts for any unsaved changes, closes the workspace, and returns to the launcher.

4.2. Tools Menu

Menu item (Tools)	Behavior and purpose
Connect an AI...	Opens the three-track setup wizard: Local AI (Ollama, LM Studio, OpenAI-compatible), Subscription CLI Agents (Claude Code, Gemini CLI, Codex, Copilot CLI, Antigravity), or Cloud API keys saved safely in the OS keychain.
Apply template...	Migrates existing document content into a chosen built-in or Typst Universe template structure without discarding your text.
MCP server...	Starts, stops, and inspects the local Model Context Protocol endpoint so external AI tools can compile and inspect the project safely.
Typst terminal...	Opens the built-in floating console (`>_`) running direct official Typst compiler commands (`compile`, `fonts`, `query`, `--version`).

Menu item (Tools)	Behavior and purpose
Run Python script…	Runs a `.py` script from the project (e.g., regenerating Matplotlib charts or CSV datasets), streaming `stdout`/`stderr` and triggering immediate preview recompilation.
Diagram editor…	Interactive vector canvas that exports clean `cetz` / `fletcher` diagram blocks.
Equation editor…	Visual math palette with live Typst-compiled preview and a **Paste LaTeX** converter powered by `MiTeX`.
Sequence diagram…	Structured participant and message builder targeting the `chronos` package.
Gantt chart…	Task timeline builder with real start/end dates compiled via `gantty`.
Kanban board…	Visual columns and cards builder (with assignee, difficulty, and priority) targeting `kantan` (AGPL-3.0-only).
Data viewer (CSV/TSV)…	Tabular inspector with delimiter detection, numeric/text sorting, column filtering, live `csv()` insertion, and AI data chat.
DOT / Graphviz…	Raw DOT graph editor featuring real-time Typst preview compiled with `diagraph`.
Edit global snippets…	Opens your cross-project personal `snippets.code-snippets` file.
Import Sublime snippets…	Converts `.sublime-snippet` files or `Packages/User` folders into VS Code `.code-snippets` format with a preview summary.
Edit project snippets…	Opens `.vscode/*.code-snippets` inside the workspace so teams can share snippets via Git.
Insert JS script (jogs)…	Inserts a ready-to-run `@preview/jogs` snippet to execute deterministic JavaScript helpers inline during Typst compilation.
Clone Git repository…	Clones a remote HTTPS or SSH repository into a local folder and opens it as a Typst workspace.

4.3. Preferences Menu

Preference	Effect and persistence
Show hidden files	Shows or hides dotfiles/folders (`.git`, `.vscode`) in the Files tree and allows project search to scan them.
Line numbers	Toggles gutter line numbers in the code editor.
Relative line numbers	Displays distances relative to the active cursor line—ideal for fast vertical motions (`12j`, `5k`) especially with Vim mode enabled.
Vim mode	Enables optional modal Vim editing (`NORMAL`, `INSERT`, `VISUAL`, `V-LINE`, `V-BLOCK`, `REPLACE`) loaded lazily on demand, showing a footer mode badge, `:w` save support, and floating-panel-safe `Escape` handling.
Auto-save	Automatically persists modified buffers after brief typing pauses.
Show full path	Displays the full file path in the document bar to disambiguate identically named files across chapters.
Ask before updating references	When moving or renaming a `.typ`, `.bib`, or image asset in the Files panel, offers to rewrite all affected `#include`, `#import`, `image()`, and `bibliography()` calls across the project.
Local version history	Saves automatic snapshots before bulk search-and-replace or assisted edits so changes can be undone independently of Git.
Share editor state with MCP agents	Controls whether connected external MCP clients may inspect cursor position, selection, or diagnostics.

Preference	Effect and persistence
Themes & colors…	Opens the theme gallery (10 curated themes), configures the 3 header quick-switch buttons, and edits a live Custom palette with WCAG AA contrast verification.
Clear local history…	Clears cached local project snapshots after confirmation to reclaim disk space.

5.

High-Performance Authoring, Toolbar & Vim Mode

5.1. Context-Aware Toolbar & BibTeX Citations

Above the editor, quick buttons insert **bold**, *italic*, strikethrough, inline code, superscript, subscript, headings (`H1`–`H3`), lists, links, notes, code blocks, tables, and figures. Placing the cursor inside an equation `$...$` automatically dims prose formatting controls and promotes math actions to the front.

For citations and cross-references:

- Type `@` anywhere in `.typ` files for a filterable completion list covering figures (`@fig:...`), tables (`@tab:...`), equations (`@eq:...`), headings, and `bibliography.bib` entries.
- Click **Cite** on the toolbar to search references by author, title, or year, or create a brand-new BibTeX entry via the built-in modal without leaving your draft.

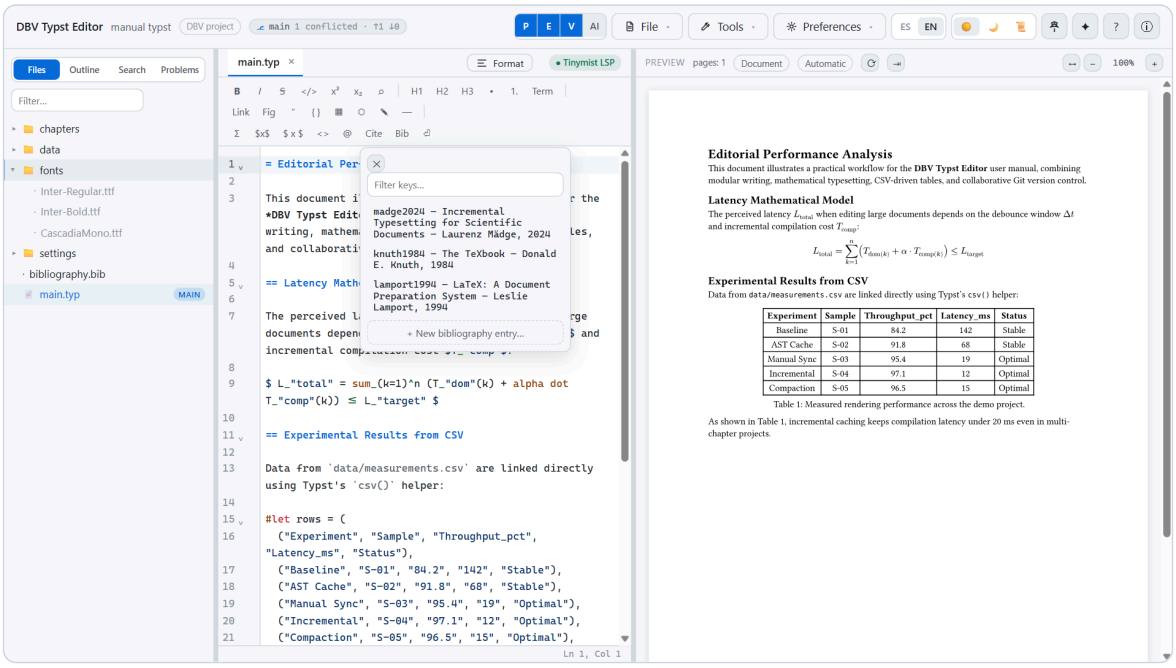


Figure 5: Interactive BibTeX citation picker triggered directly from the editor toolbar.

5.2. Optional Vim Mode, Relative Lines & Zero-Latency Manual Refresh

Authors working on book-length manuscripts and Vi/Neovim users benefit from three complementary features in **Preferences** and the preview bar:

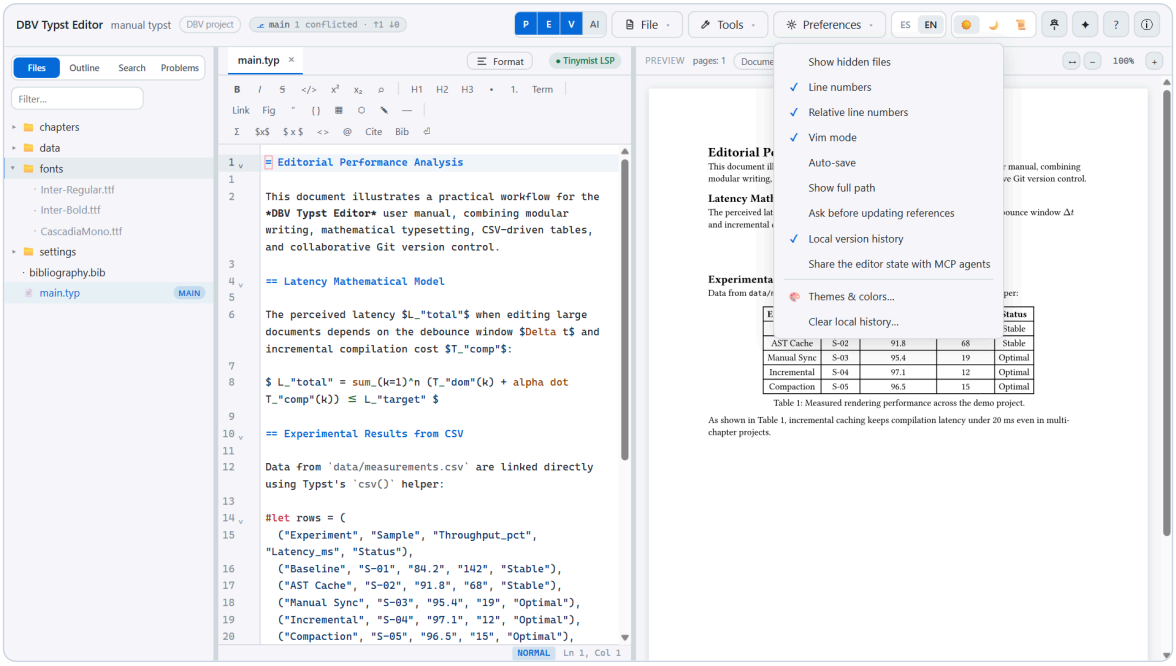


Figure 6: Preferences menu with Relative line numbers and Vim mode enabled; the bottom status bar displays the live NORMAL pill next to Ln 1, Col 1.

1. **On-demand Vim mode:** keeping `Preferences › Vim mode` unchecked incurs zero startup overhead. Turning it on shows `NORMAL`, `INSERT`, `VISUAL`, `V-LINE`, `V-BLOCK`, or `REPLACE` in the status bar, binds `:w` to save, and intercepts `Escape` cleanly so returning to Normal mode never closes open popovers or the AI panel.
2. **Relative line numbers:** makes vertical jumps effortless (`12j`, `5k`, `d8k`) by numbering lines relative to the cursor.
3. **Manual Preview Refresh:** for massive theses packed with high-res figures, toggling **Automatic** to **Manual** in the preview bar decouples typing from continuous compiles—LSP edits debounce and open tabs refresh incrementally for near-zero typing latency. Click ⟳ whenever you want to render an updated preview.

6.

Visual Customization: Themes, Colors & WCAG Contrast

Open `Preferences › Themes & colors…` to customize every aspect of workspace appearance and Typst syntax highlighting.

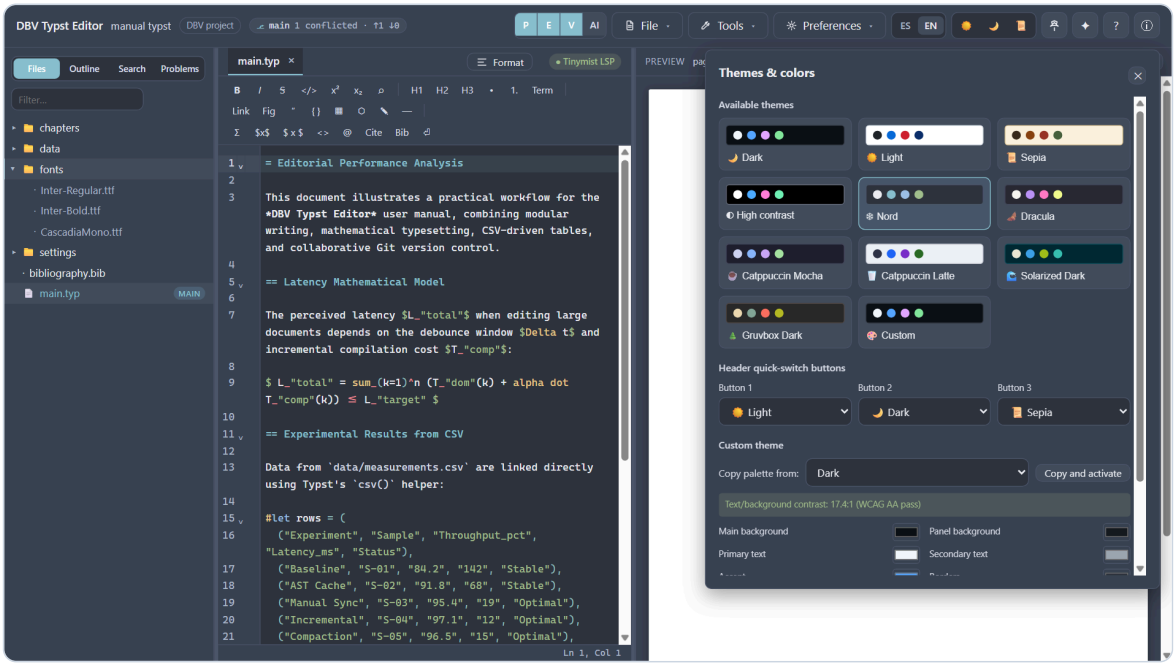


Figure 7: Themes & colors dialog showcasing Nord: 10 curated palettes, 3 configurable header quick buttons, and a live WCAG AA custom theme editor.

- **10 curated themes: Light**, **Dark** (with upgraded dark syntax contrast), **Sepia**, **High contrast**, **Nord**, **Dracula**, **Catppuccin Mocha**, **Catppuccin Latte**, **Solarized Dark**, and **Gruvbox Dark**.
- **3 header quick-switch buttons:** bind **Button 1**, **Button 2**, and **Button 3** to any curated theme or your **Custom** theme to flip effortlessly between daylight, dark, and warm reading modes.
- **Custom palette & WCAG AA meter:** click **Copy palette from…** to clone any curated theme and fine-tune backgrounds, foregrounds, accent, borders, and Typst syntax colors while a live badge reports whether foreground contrast passes **WCAG AA**.

7.

Collaborative Workflows with Git

DBV Typst Editor provides an integrated, approachable Git workflow designed for co-authored papers, lab groups, and thesis supervision inside shared repositories.

7.1. Cloning & Organizing a Shared Team Project

- Use **Clone repository** from the launcher or `Tools › Clone Git repository…` with an HTTPS/SSH remote URL and destination folder (supporting optional shallow depth for large repositories).
- Standardize the compilation root across every team member with `File › Set as main document`. This records the shared root inside `settings/dbv-project.toml` so nobody wastes time accidentally compiling isolated subchapters.
- Keep `Preferences › Ask before updating references` enabled when reorganizing chapters or assets: DBV automatically offers to update every affected `#include`, `#import`, `image()`, and `bibliography()` path across the repo.

7.2. Header Git Pill & Status Popover

Whenever the open folder contains a `.git` repository, the header pill displays:

- The active **branch** (e.g., `main`).
- Active conflict count whenever a merge conflict exists (`1 conflicted`).
- Remote divergence counters (`↑N` local commits ahead / `↓N` remote commits behind).

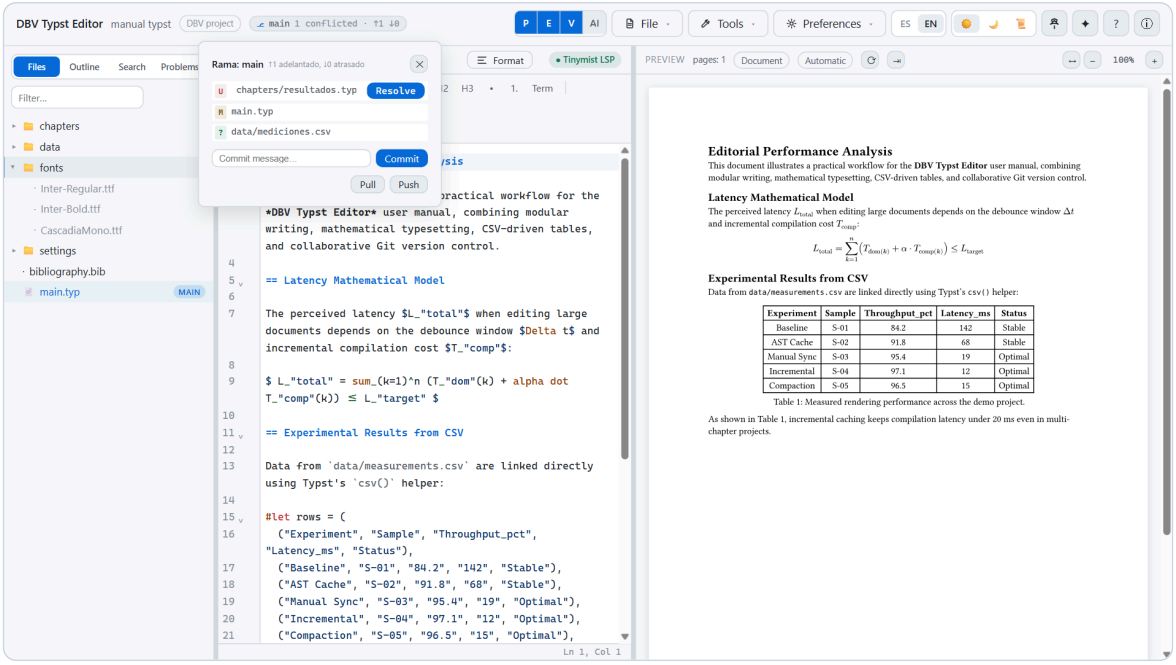


Figure 8: Header Git popover open: conflicted file (U) prioritized with a blue Resolve button, modified file (M), untracked file (?), plus Commit / Pull / Push controls.

Inside the popover, each changed file carries a clear status badge:

- `U` (highlighted blue **Resolve** action at the top): file containing unresolved conflict markers.
- `M`: tracked file with local modifications.
- `?`: untracked file newly added to the project.

Enter a message in **Commit message…** and press **Commit** to record a snapshot, or synchronize with teammates via **Pull** and **Push**.

> **Built-in Pull Overwrite Protection**

> If you trigger **Pull** while uncommitted local edits touch files modified on the remote branch, DBV blocks the merge cleanly and warns you to commit your local changes first rather than letting remote changes overwrite your draft.

7.3. Visual Block-by-Block Conflict Resolver & Disk Diff Dialog

If a **Pull** or branch merge creates conflict markers (<<<<<<<, =======, >>>>>>>), you never have to hand-edit raw markers inside Typst files:

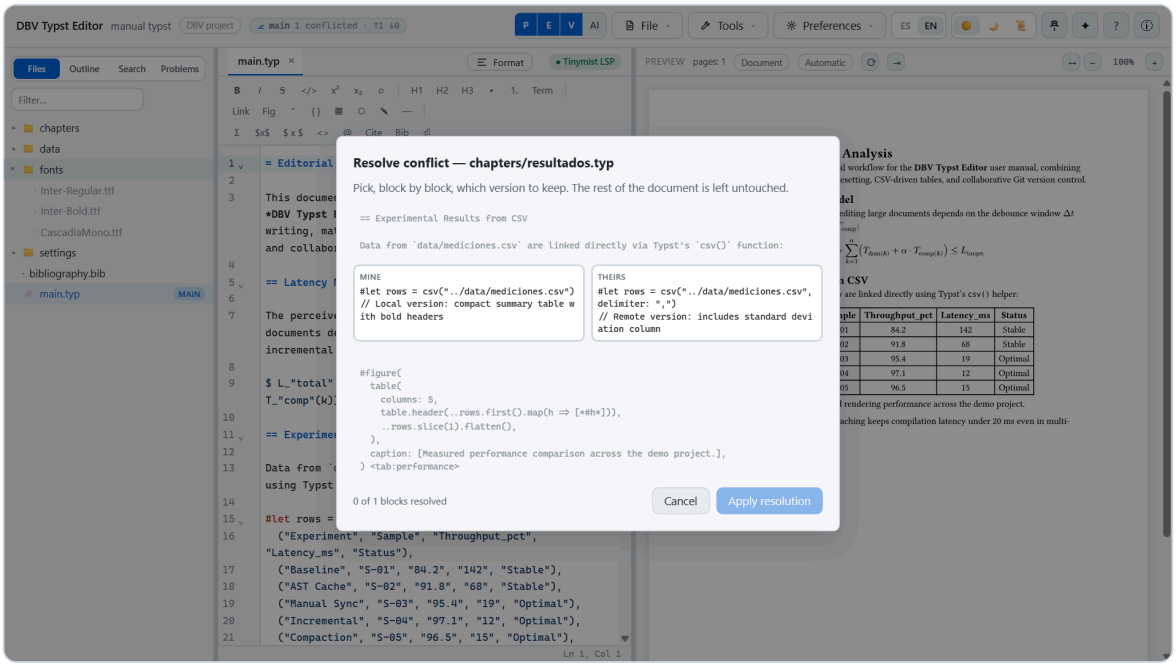


Figure 9: Visual block-by-block merge conflict resolver: compare Mine vs Theirs side by side while preserving surrounding Typst code verbatim.

1. Click the blue **Resolve** button beside any `U` entry in the Git popover.
2. The resolver isolates each conflict hunk while leaving the rest of your Typst document untouched.
3. For every block, pick **Mine** (your local `HEAD`) or **Theirs** (the incoming remote branch) as the footer tracks `N of M blocks resolved`.
4. Once all blocks are picked, press **Apply resolution** to write a clean, compilable file.

Similarly, if another tool or collaborator modifies an open file on disk while you hold unsaved edits, attempting to save opens the **side-by-side disk diff dialog** so you can keep your buffer, reload from disk, or review changes safely.

8.

Visual Tools & Assistants with Concrete Typst Examples

All specialized assistants live under **Tools** (and core authoring helpers also sit directly on the editor toolbar), generating idiomatic, human-editable Typst code.

8.1. Visual Diagram Editor (cetz)

Drag-and-drop vector canvas supporting rectangles, rounded boxes, ellipses, decision diamonds, triangles, and hexagons, arrow connectors, six colors, zoom/pan controls, and one-click seed layouts (**Flowchart**, **Blocks**, **Flow with decision**).

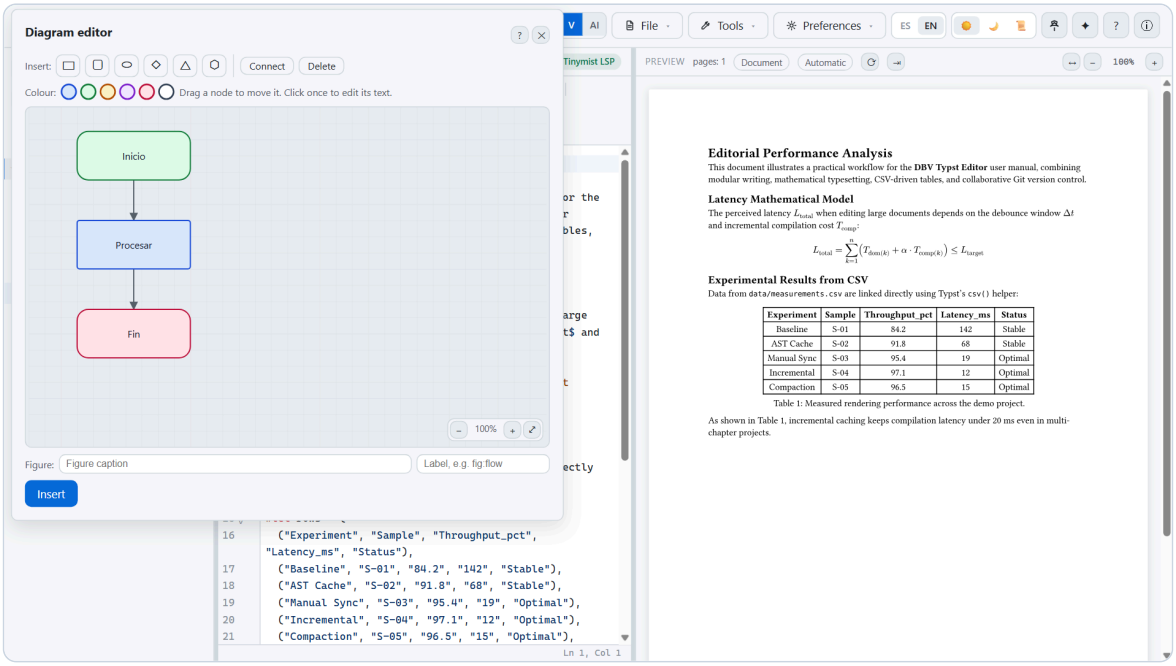


Figure 10: Visual Diagram editor seeded with a Flowchart layout ready to relabel, recolor, and insert as cetz code.

Clicking **Insert** adds the required import and figure block cleanly:

```
#import "@preview/cetz:0.5.2"
#figure(
  cetz.canvas({
    import cetz.draw: *
    // Visually arranged nodes and arrows
    rect((0, 0), (3, 1), name: "start", fill: rgb("#dcfce7"))
    content("start", [Start])
  }),
  caption: [Editorial validation pipeline.],
) <fig:pipeline>
```

8.2. Visual Equation Editor & LaTeX Converter (MiTeX)

Provides structured palettes (fractions, powers, sums, integrals, matrices, Greek letters, delimiters) paired with a **real Typst-compiled preview** underneath the input field. Have formulas from an older LaTeX manuscript? Click **Paste LaTeX** to translate via MiTeX:

```
// Inserted or converted from LaTeX into a numbered Typst equation:
$ L_"total" = sum_(k=1)^n (T_"dom"(k) + alpha dot T_"comp"(k)) <= L_"target" $
<eq:latency>
```

8.3. Sequence Diagrams (`chronos`)

Add participants and order synchronous calls (->) or dashed replies (-->) without calculating manual coordinates:

```
#import "@preview/chronos:0.2.1"
#figure(
  chronos.diagram({
    import chronos: *
    _par("Editor"); _par("LSP"); _par("Typst")
    _seq("Editor", "LSP", "didChange (debounced)")
    _seq("LSP", "Typst", "incremental compile", dashed: true)
  }),
  caption: [Non-blocking incremental compilation sequence.],
) <fig:sequence>
```

8.4. Gantt Charts (`gantty`)

Specify task names alongside real start and end dates for engineering timelines or thesis schedules:

```
#import "@preview/gantty:0.5.1"
#gantty.gantt((
  tasks: (
    (name: "SDD Specification", start: "2026-09-01", end: "2026-09-18"),
    (name: "Core & Tests", start: "2026-09-19", end: "2026-10-15"),
    (name: "User Manual & Docs", start: "2026-10-16", end: "2026-11-10"),
  )
))
```

8.5. Kanban Boards (`kantan`)

Manage columns, cards, assignees, difficulty, and priority. Targets the community `kantan` package (remember to review its `AGPL-3.0-only` license shown in the dialog):

```
#import "@preview/kantan:0.1.0": kanban
#kanban(
  columns: (
    (title: "Backlog", cards: ((title: "EN screenshots", assignee: "Team"),)),
    (title: "Review", cards: ((title: "PDF manual", priority: "High"),)),
  )
)
```

8.6. DOT / Graphviz (`diagraph`)

Paste existing `.dot` graphs exported from external tools or draft new ones with a live Typst-rendered preview underneath:

```
#import "@preview/diagraph:0.3.1": raw-render
#raw-render(```dot
digraph pipeline {
  rankdir=LR;
  Editor -> Compiler [label="AST"];
  Compiler -> Preview [label="SVG"];
}
```)
```

### 8.7. CSV / TSV Data Viewer & Live `csv()` Tables

Open any `.csv` or `.tsv` from the project tree or via `Tools › Data viewer…` to auto-detect delimiters and headers, sort numerically or alphabetically, filter rows, and choose between live linking or static copying:
```

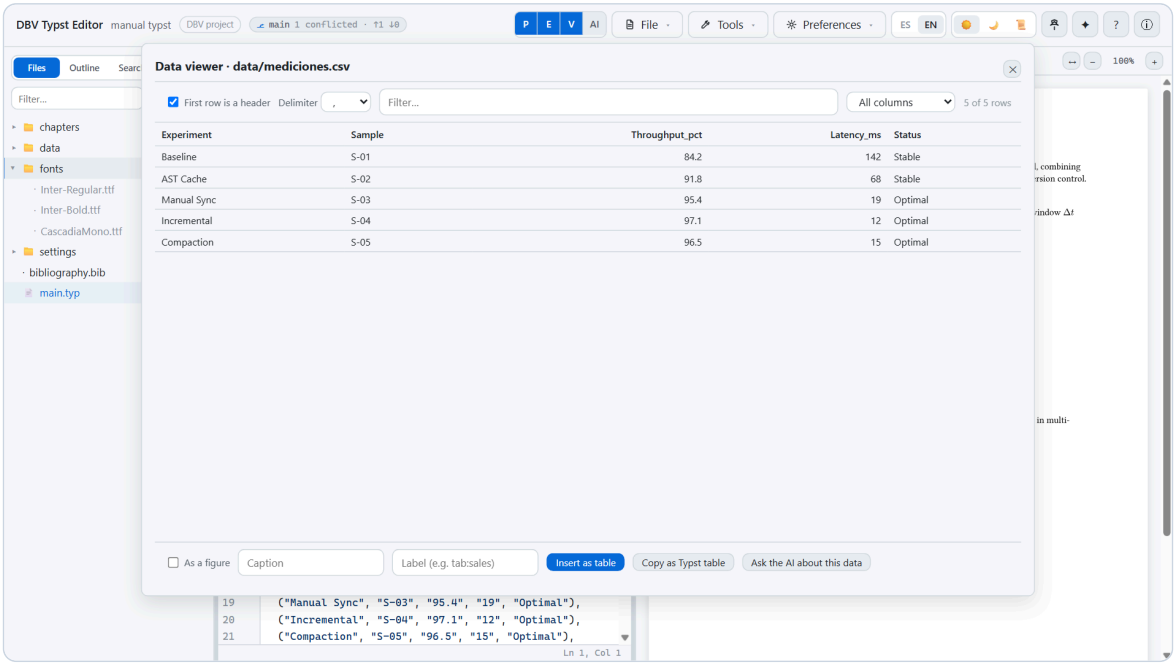


Figure 11: CSV/TSV Data viewer offering column filters, numeric sorting, live csv() table insertion, and direct AI data exploration.

- **Insert as table (best for reproducible research):** embeds a live `csv()` query so re-running experiments or scripts updates the compiled PDF table automatically:

```
#let rows = csv("data/measurements.csv")
#figure(
  table(
    columns: 5,
    table.header(..rows.first().map(h => [*#h*])),
    ..rows.slice(1).flatten(),
  ),
  caption: [Measured rendering performance loaded live from CSV.],
) <tab:performance>
```

- **Copy as Typst table:** copies a static literal table to your clipboard when you want manual cell editing detached from the file.
- **Ask the AI about this data:** forwards a tabular summary to the AI Assistant to request summary statistics, cleanup suggestions, or Typst visualization code.

8.8. Python Script Runner & Embedded JavaScript (`jogs`)

- **`Tools › Run Python script…`:** tailored for reproducible data science workflows. Pick a `.py` file inside your project (such as a Matplotlib script saving figures into `images/` or a script updating `data/*.csv`), inspect `stdout`/`stderr` directly inside DBV, and immediately watch the Typst preview update.
- **`Tools › Insert JS script (jogs)`:** inserts a `@preview/jogs` boilerplate block so you can run lightweight, deterministic JavaScript logic inside Typst compilation (e.g., array formatting or computed tabular series).

9.

VS Code Snippets, Sublime Import & Custom Keybindings

DBV Typst Editor reads standard VS Code `.vscode/*.code-snippets` and `.vscode/keybindings.json` files (with JSONC comment support) so teams can version-control reusable writing macros alongside their manuscript:

- **Global vs project scopes:** `Tools › Edit global snippets…` edits your personal cross-project file; `Tools › Edit project snippets…` saves inside `.vscode/*.code-snippets`.
- **Context menu & selection wrapping:** setting `"contextMenu": true` exposes any snippet under editor right-click (**Insert snippet ›**) and wraps selected text via `$TM_SELECTED_TEXT`.
- **Import Sublime snippets:** `Tools › Import Sublime snippets…` converts `.sublime-snippet` files or `Packages/User` folders into `.code-snippets` after showing an import summary.
- **Custom shortcuts in `.vscode/keybindings.json`:** bind snippets directly by name:

```
[
  {
    "key": "ctrl+alt+n",
    "command": "editor.action.insertSnippet",
    "args": { "name": "Theorem" }
  }
]
```

10.

Project Fonts, Typst Universe & Advanced Terminal

10.1. Self-Contained Fonts in `fonts/`

Drop `.ttf` or `.otf` font files into a root `fonts/` folder (or drag missing font files directly onto the editor). DBV passes them automatically to Typst on the very next compile with no OS installation required, and packs them into exported `.dbvt` archives.

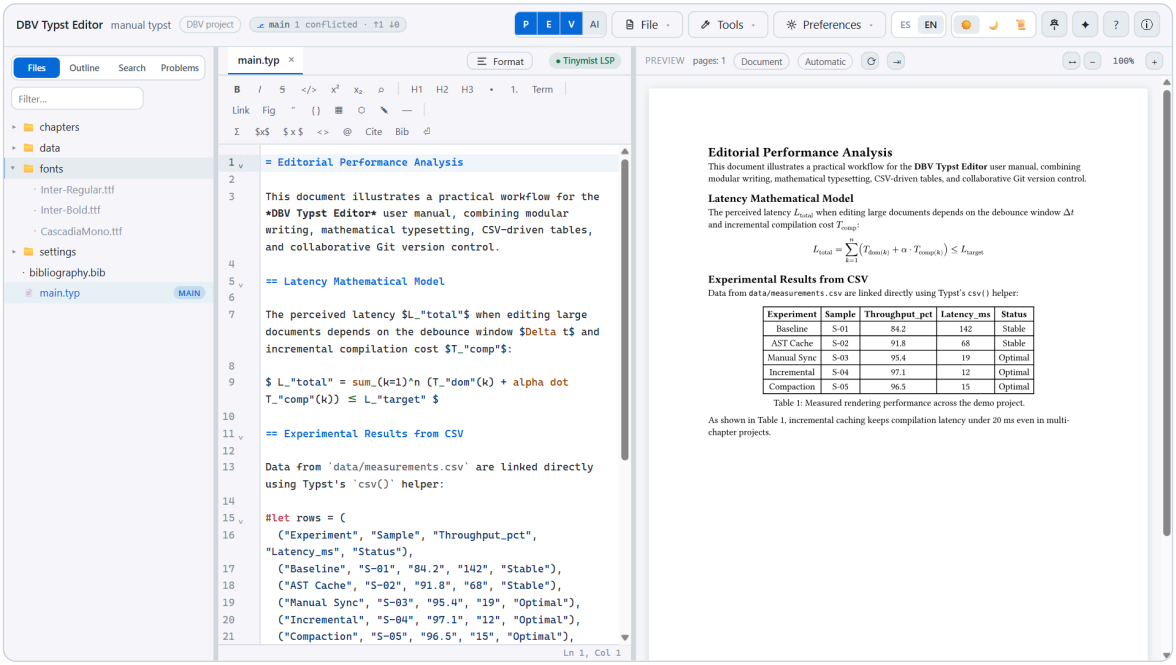


Figure 12: Self-contained fonts/ directory expanded alongside main.typ (badged MAIN), chapters/, data/, and settings/.

10.2. Typst Universe Templates & Packages

Click ✦ in the header to browse curated academic templates (IEEE, ACM `acmart`, Springer, books, `touying` slides) and popular community packages (`cetz`, `fletcher`, `codly`, `tablem`), or enter any exact Universe ID like `@preview/cetz:0.5.2`. Every entry displays its license before inserting an import at the top of the file.

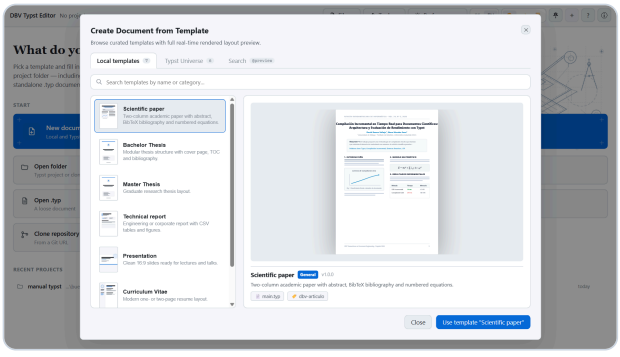


Figure 13: Combined gallery of local templates and Typst Universe community formats.

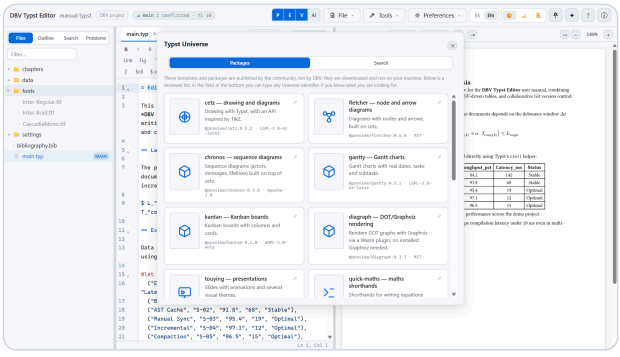


Figure 14: Typst Universe package browser showing license metadata and one-click #import insertion.

10.3. Integrated Typst Terminal & MCP Server

The >_ header button (`Tools › Typst terminal…`) opens a floating console wired directly to the bundled official Typst binary (`compile`, `fonts`, `query`, `--version`) targeting your open workspace.

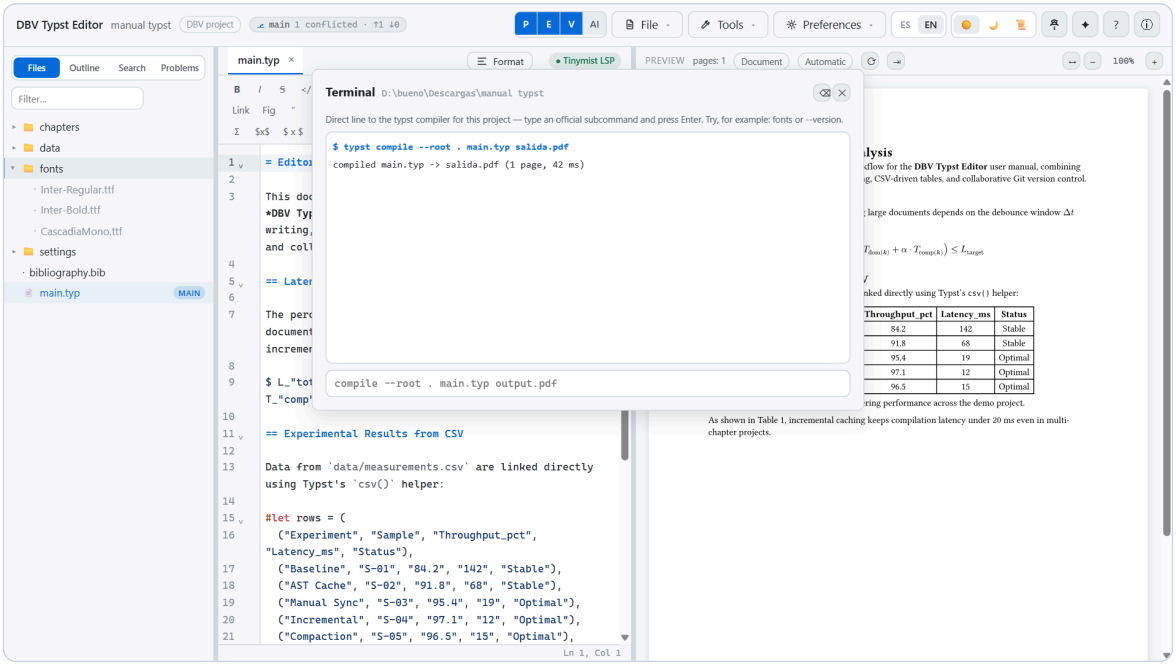


Figure 15: Floating Typst CLI terminal executing an official compile subcommand directly against the open project.

Meanwhile, `Tools › MCP server…` starts a local **Model Context Protocol** server so external coding agents can query diagnostics or trigger builds only while allowed under `Preferences › Share editor state with MCP agents`.

11.

Complete AI Assistant Guide (Local, Agents, or Cloud)

AI integration is completely **optional**: with nothing configured, DBV Typst Editor runs 100% locally and offline. From `Tools › Connect an AI…`, the wizard detects what is already installed (only when that dialog opens, never on startup) across three tracks.

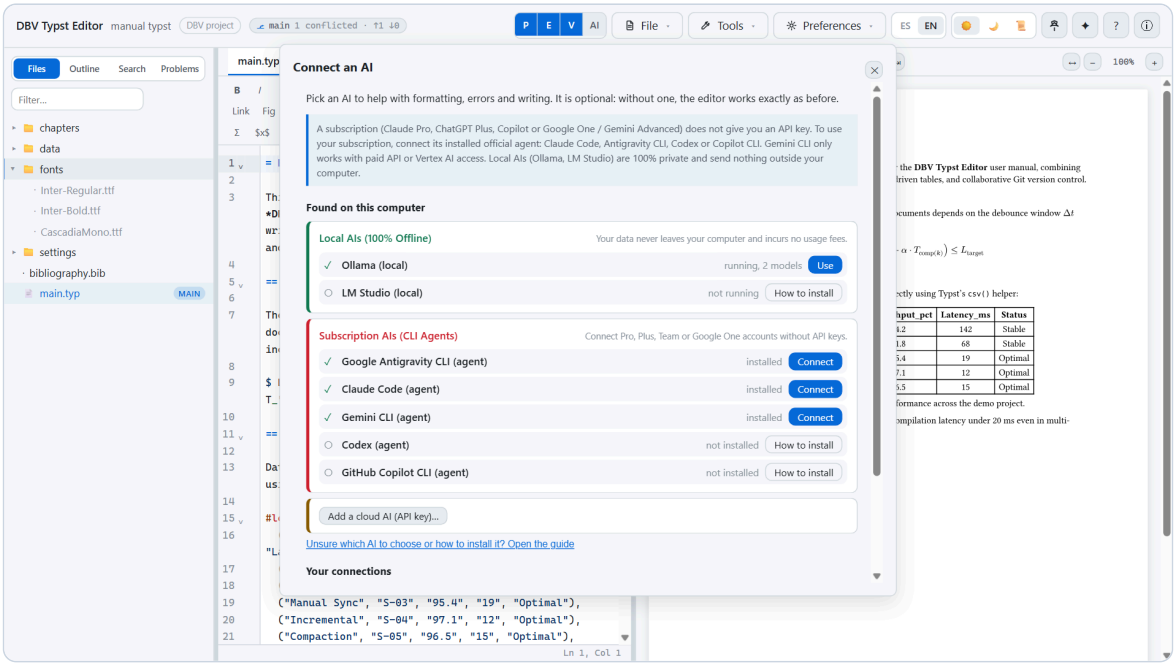


Figure 16: Connect an AI wizard organized across three tracks: Local AI (Ollama, LM Studio, OpenAI-compatible servers), CLI Subscription Agents, and Cloud API keys.

Connection route	What must be installed or configured	Leaves your PC?	Cost
Local AI (Ollama, LM Studio, `llama.cpp`)	Server running locally with at least one downloaded model.	No (100% local)	Free (uses CPU/GPU)
Cloud API key (Claude, OpenAI, Gemini, OpenRouter)	An API key issued by the provider.	Yes, context sent to provider	Pay-per-use API rates
Subscription CLI Agent (ACP protocol)	CLI agent installed & signed in: Claude Code, Codex, Gemini CLI, Google Antigravity, or GitHub Copilot CLI.	Yes, via the CLI agent	Covered by subscription

> **A subscription is NOT an API key**
>
> Paying for Claude Pro/Max, ChatGPT Plus, or Gemini does not grant an API key (API credits are billed separately). With an existing subscription, connect through the **CLI Agents** track: DBV never sees or stores your account credentials.

11.1. Track 1: Local AI on Your Machine (Total Privacy)

- **Ollama (`http://localhost:11434`):** install Ollama, pull a model (`ollama pull qwen3:8b`, inspect with `ollama list` / `ollama ps`), and click **Use** in Connect an AI. DBV speaks Ollama's native `/api/chat` protocol and manages context window size explicitly so long documents aren't silently truncated; includes an interactive **Reasoning** toggle.

- **LM Studio (`http://localhost:1234`):** start **Developer / Local Server**; context size is set inside LM Studio (**Context Length**) when loading the model, and model IDs can be inspected via `curl http://localhost:1234/v1/models`.
- **Your own OpenAI-compatible server (`llama-server`, vLLM, Jan…):** click **Add a cloud AI (API key)…** and select **OpenAI-compatible server (local or your own)** (never pick **Ollama** for a non-Ollama server). Enter an endpoint ending in `/v1` (`http://127.0.0.1:8080/v1`), the `id` from `/v1/models`, and real context (`n_ctx / total_slots` from `/props`). On Windows PowerShell use `curl.exe` instead of `curl`.

Three common `llama-server` pitfalls:

1. Your active context comes from `-c`, **not** `n_ctx_train`.
2. Multiple parallel slots (`-np`) divide the context (`-c 32768 -np 4` leaves 8,192 per request); pass `-np 1` to dedicate all tokens to your draft.
3. "Thinking" models emitting `reasoning_content` before `content` can appear silent; start `llama-server` with `--reasoning-budget 0` (e.g., `llama-server -m model.gguf -c 32768 -np 1 --reasoning-budget 0 --port 8080`).

Realistic local expectations: 4,096 tokens is the bare minimum, 8,192 is comfortable, and 32,768 is roomy. Very small local models (2–3B) often struggle with multi-file edit proposals; even with stronger 8B models such as `qwen3:8b`, always inspect proposed changes.

11.2. Track 2: Cloud Providers with an API Key

Provider	Key portal	Preconfigured endpoint
Anthropic (Claude)	Anthropic console	`https://api.anthropic.com/v1`
OpenAI (ChatGPT)	OpenAI platform	`https://api.openai.com/v1`
Google (Gemini)	Google AI Studio	`https://generativelanguage.googleapis.com/v1beta/openai`
OpenRouter	openrouter.ai	`https://openrouter.ai/api/v1`

API keys live strictly in the **OS credential store** (Windows Credential Manager, macOS Keychain, or Linux Secret Service)—never in plaintext files or `localStorage`. DBV asks for explicit confirmation the first time a project shares context with a cloud provider.

11.3. Track 3: CLI Agents Using Your Subscription (ACP)

CLI Agent	Requirements & setup	ACP launch command
Claude Code	Claude Code on `PATH` + Node.js (`npx`). Sign in once via terminal.	`npx -y @agentclientprotocol/claude-agent-acp`
Codex (OpenAI)	Codex on `PATH` + Node.js (`npx`). Sign in with ChatGPT.	`npx -y @agentclientprotocol/codex-acp`
Gemini CLI	Gemini CLI installed and authenticated in terminal.	`gemini --acp`
Google Antigravity	`agy` CLI + official `agy_acp_server` unpacked into `%LOCALAPPDATA%\agy\acp-server` (Windows) or `~/.local/share/agy-acp-server` (macOS/Linux).	`agy_acp_server`
GitHub Copilot CLI	Real CLI (`winget install GitHub.Copilot`, `brew install --cask copilot-cli`, or `npm install -g @github/copilot`). VS Code's shim `copilot` command is not enough.	`copilot --acp`

Once connected, the ⚙️ button selects the agent **model** and **reasoning effort**, while 🔄 refreshes live model lists. Each conversation keeps its own ACP session scoped to the project folder. Before each

turn, DBV saves a **restore point**: every tool request asks explicit permission (**Allow once**, **Always in this conversation**, or **Deny**) and direct disk writes surface with per-file and bulk **Undo**.

11.4. Project Capabilities & Verified Change Proposals

- **Typst-aware tools:** reads/searches within the project directory only; queries offline Typst 0.15.1 docs; compiles proposals in memory before applying; looks up real Typst Universe compatibility without guessing versions; cites valid `.bib` keys; inspects up to 3 rendered pages (vision models); and proposes missing fonts after displaying license info prior to copying into `fonts/`.
- **Changing document format ("Apply template…"):** works both without AI (`Tools › Apply template…`, preserving content, moving title/authors/abstract/keywords, and flagging `#set`/`#show` conflicts) and AI-assisted (**Let the AI adapt it**).
- **Review before writing:** every AI proposal displays compiler status (`✓ Compiles without new errors`) with **Apply selected**, **Reject**, and **Show in preview**.

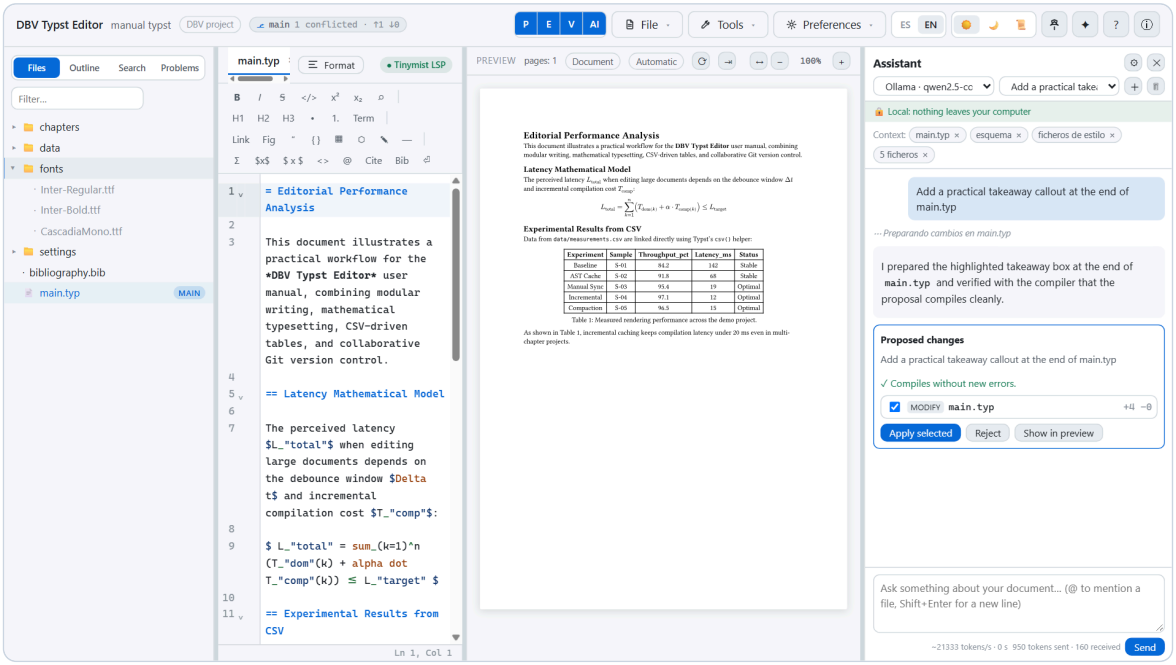


Figure 17: AI Assistant panel presenting a compiler-verified proposed change tree with Apply selected, Reject, and Show in preview actions.

11.5. Privacy Summary & Quick Troubleshooting

Common symptom	Recommended fix
No reply on custom server	Switch provider from **Ollama** to **OpenAI-compatible server** and ensure URL ends in `/v1`.
Empty reply after token usage	Thinking output hides final text; disable **Reasoning** or pass `--reasoning-budget 0` to `llama-server`.
Copilot or Antigravity unavailable	Copilot requires the standalone CLI (not only VS Code's shim); Antigravity requires both `agy` and `agy_acp_server`.
Model forgets earlier text	Increase `-c` / **Context (tokens)** or set `-np 1`.

11.6. DBV's MCP Server for External Coding Agents (`Tools › MCP server…`)

In addition to wiring into DBV's own AI panel, DBV ships a read-only **Model Context Protocol** server so external agents (terminal Claude Code, Codex CLI, Claude Desktop, Cursor, etc.) can leverage DBV's bundled Typst engine:

```
claude mcp add dbv -- "<DBV-path>" --mcp --project "<project-folder>"
codex  mcp add dbv -- "<DBV-path>" --mcp --project "<project-folder>"
```

Available tools include `compile_project`, `render_page` (up to 3 PNGs), `search_typst_docs`, `read_typst_docs`, `search_universe`, `read_package_docs`, `install_package` (gated by DBV confirmation dialog), `list_fonts`, `list_bibliography`, `citation_styles`, `editor_state`, and `dbv_info`. Enabling `Preferences › Share editor state with MCP agents` (off by default) displays an interactive **Agent connected** × cut-off pill in the header. Runs as a child process with zero open network sockets and a per-run 256-bit local token.

12.

Credits & Acknowledgments

12.1. Author & Collaborators

- **David Bueno Vallejo:** original idea, architecture, project direction, and development (`https://github.com/davidbuenov/dbv-typst-editor`, MIT License).
- **Juan Falgueras Cano:** project collaborator.
- **Alberto Corbi:** project collaborator and co-author of the reference survey on the scientific Typst ecosystem (Voynov, Corbi, López-Oliver & Gil, 2026, **IJIMAI**).

12.2. Community Contributors (`v0.15.0`)

- **`@rgranc`** (GitHub Issues #3 and #4), for detailed workflow proposals covering floating-panel stacking, menu exclusivity, user snippet keybindings, and project export behavior.
- **Víctor Buendía** (Fediscience), whose real-world testing and suggestions inspired zero-latency **Manual Refresh** optimizations, optional **Vim mode**, improved dark-theme syntax contrast, and expanded curated theme customization.

12.3. Open-Source Projects Powering DBV

- **Engine, language & sync: Typst** (Apache-2.0, bundled 0.15.1 sidecar and in-process crates `typst`, `typst-ide`, `typst-layout`, `typst-svg`, `typst-kit`), **Hilbert Editor** (MIT, whose `Span` click→source sync inspired the preview map), and **Tinymist** (Typst's official Language Server).
- **Typst Universe packages behind visual tools:** `cetz`, `MiTeX`, `Chronos`, `Gantty`, `Kantan`, `Diagraph`, and `jogs`, alongside the maintainers of every curated template and package in **Typst Universe** (IEEE, ACM, Springer, `fletcher`, `touying`, `quick-maths`, `physica`, `codly`, `zebraw`, `showybox`, `tablem`, `subpar`, `lovelace`, `glossarium`, `unify`, `wordometer`, and more).
- **Core stack & libraries: Tauri v2** (`shell`, `dialog`, `updater`, `process`, and `single-instance` plugins), **CodeMirror 6**, `codemirror-lang-typst`, `@replit/codemirror-vim`, **Hayagriva**, **Vite**, `jsonc-parser`, Rust crates (`serde`, `tokio`, `notify`, `toml`, `toml_edit`, `tempfile`, `zip`, `dunce`, `base64`, `regex`, `walkdir`, `ureq`, `sys-locale`), and **Vladimir Pankratov**'s `tauri-windows-bundle` for Microsoft Store `.msix` packaging.
- **Built with AI pair programming & SDD: Gemini** (Google DeepMind), **Claude Code** (Anthropic), **Microsoft Copilot** (Microsoft), and the open **dbv-specs-ops** framework.